Slime Mould Algorithm: formulação matemática, representação e configuração dos testes

Wendson Menezes Dias
Ciência da Computação – Universidade Federal do Maranhão (UFMA)
Disciplina: Sistemas Inteligentes
Professor: Alexandre Cesar Muniz de Oliveira

1. Inspiração natural e operadores

O SMA abstrai a busca por alimento de *Physarum polycephalum*, um organismo unicelular que forma redes de tubos com fluxo citoplasmático. A população computacional é um conjunto de soluções candidatas, não é uma simulação biológica detalhada nem exige indivíduos fisicamente separados na natureza.

Aproximação do alimento: A melhor solução encontrada faz uma orientação de atualização perturbada por posições da população. O termo v_b controla a amplitude aleatória da perturbação, não impõe trajetórias elípticas.

Envolvimento do alimento e feedback: Mudanças de fluxo e espessura dos tubos modificam os pesos W, dependentes da aptidão e da classificação dos agentes. Esses pesos regulam a influência de uma posição escolhida, em vez de afetar a etapa inteira. $W > 1$ não garante maior deslocamento nem melhor resultado.

Oscilação e exploração: v_b e v_c são sorteados em intervalos de amplitude decrescente. A reinicialização com probabilidade $z = 0{,}03$ introduz exploração. Esses mecanismos reduzem o risco de estagnação, mas não garantem encontrar o ótimo global. Não se deve apresentar a reinicialização como prova de separação física de uma parte do organismo.

2. Formulação matemática dos operadores

O Slime Mould Algorithm (SMA) é uma meta-heurística inspirada na busca por alimento de *Physarum polycephalum*. Considera-se a otimização contínua por minimização, conforme Li et al. (2020) e as convenções da implementação de referência [1, 2].

Cada solução é um vetor $X_i = (x_{i,1}, \ldots, x_{i,D})$. Na iteração t, avalia-se $S_i = f(X_i)$ e ordenam-se as aptidões: $s_k = S_{\pi(k)}$, em ordem crescente. Assim, $\pi(k)$ identifica o agente na posição k do ranking, $bF = s_1$ e $wF = s_N$. A melhor posição histórica é X_b, com aptidão DF.

Pesos adaptativos. Representam o feedback associado à qualidade e à classificação das soluções. Para $wF \neq bF$, calculam-se:

$$q_k = \frac{s_k - bF}{wF - bF}, \qquad W_{\pi(k),d} = \begin{cases} 1 + u_{k,d} \log_{10}(1 + q_k), & k \leq \lfloor N/2 \rfloor, \\ 1 - u_{k,d} \log_{10}(1 + q_k), & k > \lfloor N/2 \rfloor, \end{cases} \tag{1}$$

com $u_{k,d} \sim U(0, 1)$ independente por coordenada. Se $wF = bF$, adota-se $q_k = 0$ e $W = 1$: proteção numérica explícita para o caso indefinido, diferente da soma de `eps` utilizada na demo MATLAB.

Controles da movimentação. Para $t = 1, \ldots, T$, definem-se:

$$\begin{gathered} a(t) = \operatorname{arctanh}(1 - t/T), \quad b(t) = 1 - t/T, \quad p_i = \tanh(|S_i - DF|), \\ v_{b,i,d} \sim U(-a, a), \qquad v_{c,i,d} \sim U(-b, b). \end{gathered} \tag{2}$$

Iniciar em $t = 1$ evita $\operatorname{arctanh}(1)$. As amplitudes diminuem ao longo da execução, mas os valores sorteados não são monotônicos.

Atualização e reinicialização. Sorteia-se $g_i \sim U(0, 1)$. Se $g_i < z$, com $z = 0{,}03$, aplica-se $X_i \leftarrow LB + u(UB - LB)$, usando um novo escalar $u \sim U(0, 1)$, como na demo. Caso contrário,

para cada coordenada:

$$x_{i,d} \leftarrow \begin{cases} x_{b,d} + v_{b,i,d}(W_{i,d}\, x_{A,d} - x_{B,d}), & r_{i,d} < p_i, \\ v_{c,i,d}\, x_{i,d}, & r_{i,d} \geq p_i, \end{cases} \tag{3}$$

com novos $r_{i,d} \sim U(0,1)$ e índices A, B uniformes em $\{1, \ldots, N\}$, com reposição. Eles podem coincidir entre si ou com i. A primeira alternativa perturba a melhor posição; a segunda contrai em direção à origem, com possível inversão de sinal. As atualizações são sequenciais, na própria matriz X, mantendo S, W e DF fixos nessa etapa. Reinicializar com um vetor de sorteios independentes seria uma adaptação da linha escalar da demonstração.

3. Entradas, saídas e representação das soluções

Entradas. Função objetivo $f : \Omega \to \mathbb{R}$; dimensão D; limites finitos $LB_d < UB_d$; população N; número de iterações T; parâmetro z e semente do gerador aleatório. O domínio é $\Omega = \prod_{d=1}^{D} [LB_d, UB_d]$, com N, D, T inteiros positivos.

Representação. A população é a matriz real $X \in \mathbb{R}^{N \times D}$: cada linha representa um indivíduo, e cada coluna, uma variável de decisão. O estado também contém $W \in \mathbb{R}^{N \times D}$, o vetor de aptidões $S \in \mathbb{R}^N$, a permutação π, o vetor $X_b \in \mathbb{R}^D$ e o escalar DF. Não há codificação binária.

Inicialização e fronteiras. Sorteios independentes por indivíduo e coordenada distribuem a população uniformemente no hiperetângulo. Antes de cada avaliação, posições fora do domínio são projetadas nos limites:

$$\begin{gathered} x_{i,d}^{(0)} = LB_d + u_{i,d}(UB_d - LB_d), \qquad u_{i,d} \sim U(0,1), \\ x_{i,d} \leftarrow \min\{UB_d, \max\{LB_d, x_{i,d}\}\}. \end{gathered} \tag{4}$$

Fluxo e saídas. Inicializa-se $DF = +\infty$. Em cada iteração: corrigir fronteiras, avaliar a população, ordenar aptidões, calcular pesos, atualizar a cópia independente do melhor histórico e movimentar os agentes. Retornam-se X_b, $DF = f(X_b)$ e a curva $C(t) = DF(t)$. A população pode piorar, mas o melhor histórico é preservado. Uma última movimentação não é avaliada no fluxo da demonstração, portanto, não pode ser declarada uma melhoria e as saídas não garantem um ótimo global.

4. Fitness e espaço de busca dos testes

A aptidão é diretamente o valor da função objetivo, sem transformação: quanto menor $f(x)$, melhor a solução. Selecionam-se três funções com $D = 30$; os limites abaixo valem para todas as coordenadas.

$$f_{\text{Sphere}}(x) = \sum_{d=1}^{D} x_d^2, \qquad \Omega = [-100, 100]^{30}. \tag{5}$$

$$f_{\text{Rastrigin}}(x) = 10D + \sum_{d=1}^{D} [x_d^2 - 10\cos(2\pi x_d)], \quad \Omega = [-5{,}12;\ 5{,}12]^{30}. \tag{6}$$

$$\begin{aligned} f_{\text{Ackley}}(x) = & -20 \exp\left(-0{,}2 \sqrt{\frac{1}{D} \sum_{d=1}^{D} x_d^2} \right) \\ & - \exp\left(\frac{1}{D} \sum_{d=1}^{D} \cos(2\pi x_d) \right) + 20 + e, \qquad \Omega = [-32, 32]^{30}. \end{aligned} \tag{7}$$

Sphere é unimodal; Rastrigin e Ackley são multimodais, são usados radianos, todas possuem um ótimo global $x^* = 0$ e $f(x^*) = 0$, conhecido apenas para análise. Como seus ótimos estão na origem, bons resultados não demonstram desempenho geral do SMA.

Protocolo proposto. Adotam-se $N = 30$, $T = 500$, $z = 0{,}03$ e 30 execuções independentes por função, com 15.000 avaliações por execução. Devem-se registrar sementes, gerador e convenções numéricas; apresentar média, desvio padrão amostral, mediana da aptidão final e curvas $C(t)$. Essa configuração é uma proposta de teste, sem resultados experimentais neste relatório.

Referências

[1] LI, S.; CHEN, H.; WANG, M.; HEIDARI, A. A.; MIRJALILI, S. Slime mould algorithm: A new method for stochastic optimization. Future Generation Computer Systems, v. 111, p. 300–323, 2020. DOI: 10.1016/j.future.2020.03.055.

[2] SMA.m: demo versão 1.0. Pacote MATLAB, 2020. Disponível em: `https://aliasgharheidari.com/SMA.html`. Consulta: 15 out. 2026.

[3] GHAREHCHOPOGH, F. S.; UCAN, A.; IBRIKCI, T.; ARASTEH, B.; ISIK, G. Slime Mould Algorithm: A Comprehensive Survey of Its Variants and Applications. Archives of Computational Methods in Engineering, v. 30, p. 2683–2723, 2023. DOI: 10.1007/s11831-023-09883-3.